

Number of Sales per Discrete Months

➤ code

```
elif selected_operation == 'Number of Sales per Discrete Months':  
    # Calculate number of sales per discrete months  
    month = data.groupby(data['Date'].dt.strftime('%m'))['itemName'].count()  
  
    # Plot the graph  
    fig = px.bar(month, x=month.index, y=month, color=month,  
                 labels={'y': 'Number of Sales', 'Date': 'Months'})  
    fig.update_layout(title_text='Number of Sales per Discrete Months',  
                      title_x=0.5, title_font=dict(size=18))  
    fig.update_traces(marker=dict(line=dict(color='#000000', width=1)))  
    st.plotly_chart(fig)
```

Explanation:

1. Number of Sales per Discrete Months

- **Purpose:** Determine the number of items sold each month.
- **How It Works:**
 1. **Parameters Used:**
 - **Date:** Extracted in the format %m (month as "01", "02", etc.).
 - **itemName:** The item sold, whose count is calculated.
 2. **Prediction Basis:**
 - Sales are grouped by the month (Date column) to compute total sales for each month.
 - The grouped data reveals sales patterns for specific months, helping predict high-activity months based on historical trends.
 3. **Visual Analysis:**
 - The bar chart highlights months with higher or lower sales, providing insights into seasonal trends or promotional impacts.

➤ Number of Sales per Discrete Months

elif selected_operation == 'Number of Sales per Discrete Months':

- **Explanation:** Checks if the selected operation (from user input) is "Number of Sales per Discrete Months." If true, the code inside this block will execute.
- ```
month = data.groupby(data['Date'].dt.strftime('%m'))['itemName'].count()
```
- **data['Date'].dt.strftime('%m'):** Extracts the month (e.g., "01" for January) from the Date column.
  - **data.groupby():** Groups the data based on the extracted month.
  - **['itemName'].count():** Counts the number of itemName entries in each group (month).
  - **Result:** Returns a Series where the index is the month, and the values are the number of sales.

```
fig = px.bar(month, x=month.index, y=month, color=month,
 labels={'y': 'Number of Sales', 'Date': 'Months'})
```

- **px.bar**: Creates a bar chart.
  - **x=month.index**: Sets the x-axis to the month numbers (e.g., "01", "02").
  - **y=month**: Sets the y-axis to the number of sales for each month.
  - **color=month**: Color-codes the bars based on sales numbers.
  - **labels**: Customizes axis labels.

```
fig.update_layout(title_text='Number of Sales per Discrete Months',
 title_x=0.5, title_font=dict(size=18))
```

- **update\_layout**: Adds and customizes the chart title.
  - **title\_text**: Sets the chart title.
  - **title\_x=0.5**: Centers the title horizontally.
  - **title\_font**: Sets the font size of the title.

```
fig.update_traces(marker=dict(line=dict(color='#000000', width=1)))
```

- **update\_traces**: Styles the bars by adding black outlines (#000000) with a width of 1 pixel.

```
st.plotly_chart(fig)
```

- **st.plotly\_chart(fig)**: Displays the Plotly chart in the Streamlit web app.

---

## Number of Sales per Discrete Month Days

### ➤ code

```
elif selected_operation == 'Number of Sales per Discrete Month Days':
 # Calculate number of sales per discrete month days
 month_day = data.groupby(data['Date'].dt.strftime('%d'))['itemName'].count()

 # Plot the graph
 fig = px.bar(month_day, x=month_day.index, y=month_day, color=month_day,
 labels={'y': 'Number of Sales', 'Date': 'Month Days'})
 fig.update_layout(title_text='Number of Sales per Discrete Month Days',
 title_x=0.5, title_font=dict(size=18))
 fig.update_traces(marker=dict(line=dict(color='#000000', width=1)))
 st.plotly_chart(fig)
```

## 2. Number of Sales per Discrete Month Days

- **Purpose**: Analyze daily sales trends within each month.
- **How It Works**:
  1. **Parameters Used**:

- **Date:** Extracted in the format %d (day of the month as "01", "02", ..., "31").
- **itemName:** The item sold, whose count is calculated.

## 2. Prediction Basis:

- Sales are grouped by day to identify spikes or dips in daily sales activity.
- This analysis can predict specific days (e.g., weekends, holidays) with higher or lower sales volumes.

## 3. Visual Analysis:

- The bar chart displays sales activity for each day, aiding in identifying regular peak or low-traffic days.

### ➤ Number of Sales per Discrete Month Days

`elif selected_operation == 'Number of Sales per Discrete Month Days':`

- **Explanation:** Checks if the selected operation is "Number of Sales per Discrete Month Days." If true, this block executes.

`month_day = data.groupby(data['Date'].dt.strftime('%d'))['itemName'].count()`

- **data['Date'].dt.strftime('%d'):** Extracts the day of the month (e.g., "01", "02").
- **Grouping and Counting:** Similar to the first module, this groups the data by days and counts the sales (itemName).

`fig = px.bar(month_day, x=month_day.index, y=month_day, color=month_day, labels={'y': 'Number of Sales', 'Date': 'Month Days'})`

- **Creates a Bar Chart:**
  - **x=month\_day.index:** Sets the x-axis to day numbers.
  - **y=month\_day:** Sets the y-axis to sales counts for each day.
  - **color=month\_day:** Color-codes the bars based on daily sales.

`fig.update_layout(title_text='Number of Sales per Discrete Month Days', title_x=0.5, title_font=dict(size=18))`

- **Customizes the Layout:**
  - Similar to the previous module, this sets the title and aligns it centrally.

```
fig.update_traces(marker=dict(line=dict(color='#000000', width=1)))
```

- **Styles the Bars:**
  - Adds black outlines for better visibility.

```
st.plotly_chart(fig)
```

- **Displays the Chart:**
  - Renders the bar chart in the Streamlit application.

## Recency Distribution of the Customers

### ➤ code

```
elif selected_operation == 'Recency Distribution of the Customers':
 # Assuming you have a DataFrame named Recency containing the recency values
 # You need to replace Recency with your actual DataFrame name
 Recency = data.groupby(by='memberID')['Date'].max().reset_index()
 Recency.columns = ['memberID', 'LastDate']
 Recency.head()

 # Finding last date for our dataset
 last_date_dataset = Recency['LastDate'].max()
 last_date_dataset

 # Calculating Recency by subtracting (last transaction date of dataset) and (last
 purchase date of each customer)
 Recency['Recency'] = Recency['LastDate'].apply(lambda x: (last_date_dataset -
 x).days)
 Recency.head()

 # Plot the histogram
 fig = px.histogram(Recency, x='Recency', opacity=0.85, marginal='box')
 fig.update_traces(marker=dict(line=dict(color='#000000', width=1)))
 fig.update_layout(title_text='Recency Distribution of the Customers', title_x=0.5,
 title_font=dict(size=20))
 st.plotly_chart(fig)
```

### 3. Recency Distribution of Customers

- **Purpose:** Assess customer engagement by determining the recency of their last purchase.
- **How It Works:**
  1. **Parameters Used:**
    - **memberID:** Unique identifier for customers.
    - **Date:** Used to compute the last purchase date for each customer.
    - **Recency:** The number of days since a customer's last purchase.
  2. **Prediction Basis:**
    - The difference between the most recent transaction date (last\_date\_dataset) and the customer's last purchase date (LastDate) provides the Recency.
    - Customers with lower recency values are considered more active, while higher values indicate inactivity.
    - Insights can predict customer retention and re-engagement strategies, such as targeting customers with high recency for marketing campaigns.
  3. **Visual Analysis:**
    - A histogram displays the distribution of recency values across customers.
    - The optional box plot summarizes data spread, outliers, and central tendencies.

#### ➤ Recency Distribution of Customers

elif selected\_operation == 'Recency Distribution of the Customers':

- **Explanation:** Checks if the selected operation is "Recency Distribution of the Customers." If true, this block executes.

Recency = data.groupby(by='memberID')['Date'].max().reset\_index()

- **data.groupby(by='memberID'):** Groups data by memberID (each customer).
- **['Date'].max():** Finds the most recent purchase date for each customer.
- **reset\_index():** Resets the index to turn the grouped data into a DataFrame.
- **Result:** Creates a DataFrame named Recency with columns:

- memberID: Unique customer ID.
- LastDate: Most recent purchase date for each customer.

`Recency.columns = ['memberID', 'LastDate']`

- Renames columns for clarity.

`last_date_dataset = Recency['LastDate'].max()`

- Finds the last transaction date in the dataset.

`Recency['Recency'] = Recency['LastDate'].apply(lambda x: (last_date_dataset - x).days)`

- **apply()**: Calculates the difference in days between last\_date\_dataset and each customer's LastDate.
- **Result**: Adds a Recency column with the calculated number of days since the customer's last transaction.

`fig = px.histogram(Recency, x='Recency', opacity=0.85, marginal='box')`

- **Creates a Histogram**:
  - **x='Recency'**: Plots the Recency values (days).
  - **opacity=0.85**: Sets transparency of bars.
  - **marginal='box'**: Adds a box plot summarizing distribution.

`fig.update_traces(marker=dict(line=dict(color='#000000', width=1)))`

- Adds black outlines to the histogram bars.

`fig.update_layout(title_text='Recency Distribution of the Customers',  
title_x=0.5, title_font=dict(size=20))`

- Customizes the title and layout.

`st.plotly_chart(fig)`

- Displays the histogram in the Streamlit application.

---

## Summary

1. **Input:** User selects an operation (selected\_operation), and the dataset data is used for analysis.
2. **Processing:** The data is grouped and aggregated based on:
  - Months (%m) for monthly sales.
  - Days (%d) for daily sales.
  - Customer IDs (memberID) for recency calculation.
3. **Output:** Interactive visualizations (bar charts, histograms) are rendered using Plotly and displayed via Streamlit.

## General Workflow of Modules

1. **Input Parameters:**
  - Each module uses specific columns (Date, itemName, memberID) from the dataset to group, filter, and compute aggregated values.
2. **Processing:**
  - Data is grouped based on temporal dimensions (month, day) or customer-specific metrics (memberID).
  - Relevant calculations (e.g., counts, date differences) are applied to produce meaningful summaries.
3. **Prediction Insights:**
  - **Monthly Sales:** Helps predict future sales for each month by observing seasonal or promotional trends.
  - **Daily Sales:** Identifies high-performing days and predicts peak sales windows within months.
  - **Customer Recency:** Categorizes customers into active, dormant, or at-risk groups for retention strategies.

## Visualization as Decision Support

Each chart serves as a decision-support tool, helping users intuitively grasp patterns and trends that drive strategic decisions. By analyzing this data, businesses can predict market demands, plan stock replenishments, and target customer re-engagement campaigns effectively.